PROJECT REPORT · IMMERSIVE TECHNOLOGIES · A.Y. 2025–2026

Virtual Office

A Multi-User Immersive Collaborative Workspace in VR

Riccardo Zannoni
riccardo.zannoni@studenti.unitn.it
256121

Nicola Cappellaro
nicola.cappellaro@studenti.unitn.it
256119

Unity 6 | Ubiq 1.0 | Meta Quest 3 | C#

MOTIVATION

The Problem: Sequential Visual Collaboration

🔒 Sequential Screen Sharing

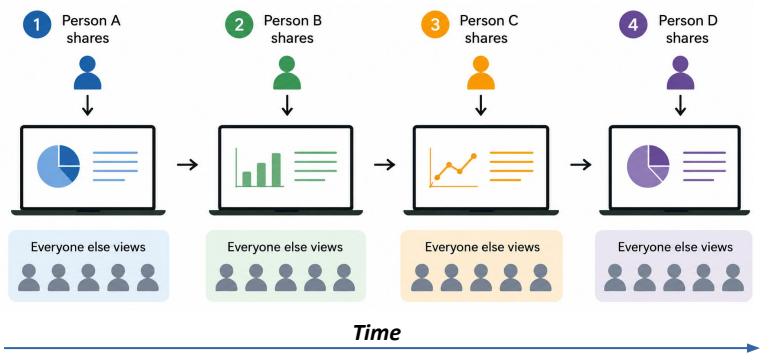
In Zoom, Google Meet and Teams only one user at a time can share their screen.

🔄 Constant Handoffs

Collaborative sessions degenerate into a series of presenter handoffs between participants.

👁️ Loss of Context

It's impossible to monitor others' work in real time, reducing the perception of a shared space.



Issue: 1 presenter at a time

SOLUTION

Virtual Office: Our Answer

A shared virtual office in VR where up to 4 remote users collaborate in real time, each at their own workstation with a personal whiteboard and virtual monitors that are always visible.

Personal Workstation

Each user has a dedicated desk, whiteboard and monitors

Digital Whiteboard

Virtual pen with HSV color picker and eraser

Always-On Visibility

Monitors show other users' whiteboards via Render Textures

Spatial Audio

Voice built in with Ubiq, no extra setup required

DESIGN

System Architecture

Hardware
Meta Quest 3 with controllers

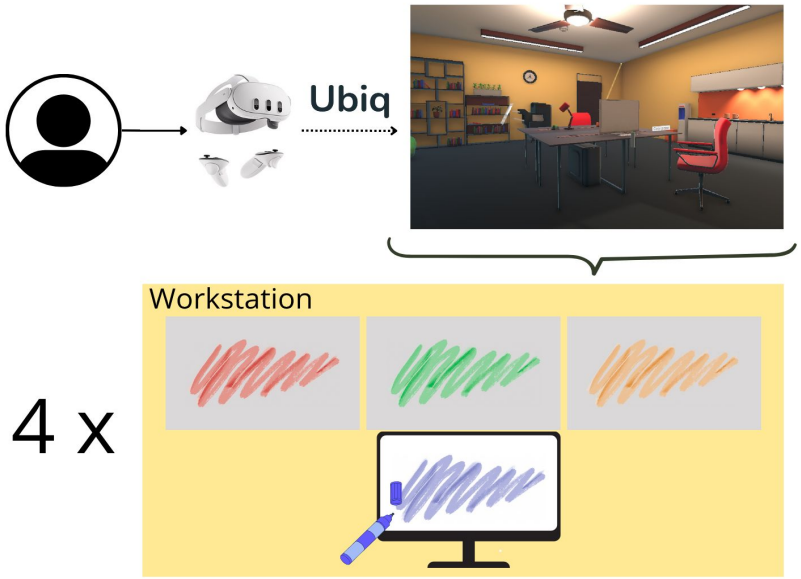
XR Interaction
XR Interaction Toolkit 3.0.7

Engine
Unity 6.0 · C# Scripts

Networking
Ubiq 1.0.0-pre16 · Local server

Assets & Tools
Office Pack · Skybox Series · VSCode

Two paths: Local (in-scene Raycast) · Networked (Ubiq NetworkContext)



System architecture

DESIGN

The Workstation: Core of the Experience

First-person view

01 **Whiteboard**

1920×1080 RGBA32 Texture2D rendered on the monitor mesh. Synchronized over the network.

02 **Virtual Monitors**

3 planes sampling the RenderTextures of other users' whiteboards with zero additional overhead.

03 **WhiteboardMarker**

Pen with an XRGrabInteractable. Raycast from the nib, linear interpolation between frames for continuous strokes.

04 **Color Picker**

Floating HSV panel with real-time preview, hex input and an Eraser button.

IMPLEMENTATION

Networking & Synchronization Protocols

Late-Join Snapshot

› The peer with the smallest UUID (master) is elected deterministically.

› The texture is JPEG-compressed, base64-encoded, and split into 24 KB chunks.

› The new peer reassembles the chunks and immediately shows the current whiteboard.

Slot Assignment

› Each peer reads the others' Ubiq properties after a short delay (coroutine).

› It picks the first free slot (1-4) and writes it to its own peer property.

› Avoids the race condition where everyone ended up in slot 1.

Stroke Interpolation

› The network message includes the current position and the last point.

› The receiving peer linearly interpolates between the two points.

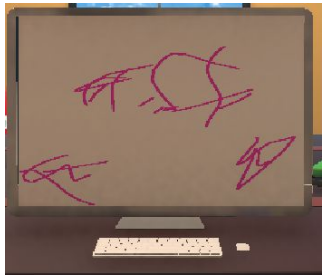
› Produces continuous strokes regardless of the network tick rate.

IMPLEMENTATION

Custom C# Scripts

{} Whiteboard.cs

Real-time stroke synchronization + late-join protocol with JPEG → base64 chunking.



{} WhiteboardMarker.cs

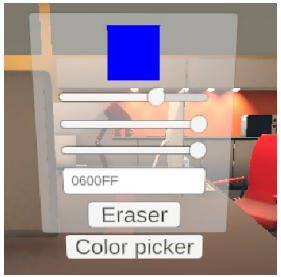
XR grab, raycast UV → pixel mapping, interpolation, owner/non-owner pattern for pen-position sync.

{} WorkstationManager.cs

Listens to Ubiq events (OnJoinedRoom, OnPeerAdded, OnPeerRemoved) + slot-assignment coroutine + dynamic visibility.

{} ColorPickerUI.cs

HSV color picker with sliders, hex input, live swatch preview and dynamic binding to the active marker.



CHALLENGES

Challenges Faced & Solutions Adopted

⚠️ **Race condition in slot assignment**

↓

✓ A coroutine delay before reading peer properties: the new peer waits for already-occupied slots to propagate.

⚠️ **Discontinuous strokes over the network**

↓

✓ Sending last_point with every message + linear interpolation on the receiver → smooth strokes at any tick rate.

⚠️ **All workstations always visible**

↓

✓ WorkstationManager dynamically activates only the workstations matching connected users.

⚠️ **Color picker with no target pen**

↓

✓ Marker ownership system: the ColorPickerUI registers the closest grabbed pen and applies color changes to it.

EVALUATION

Evaluation & Results

✅ Functional Tests Passed

- ✓ Single user: whiteboard rendering, pen grab/release, color picker
- ✓ 2 users: real-time stroke sync + Virtual_Whiteboard mirroring
- ✓ 3-4 users: workstation activation, slot assignment, correct visibility
- ✓ Late join: snapshot correctly transmitted and applied

⚠️ Known issue: on disconnect, the last workstation is deactivated instead of the one belonging to the leaving peer.

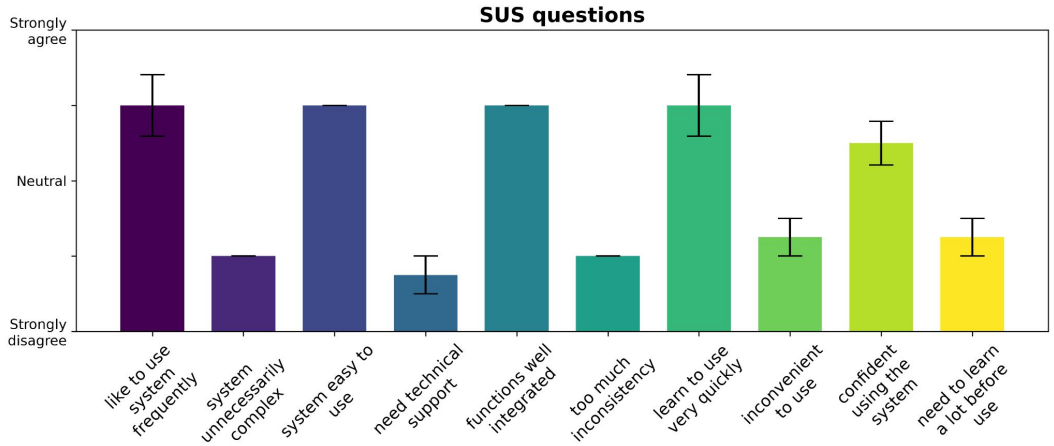
💬 Informal Usability Observations

- Experience described as genuinely immersive compared to a standard video call.
- Always-on whiteboard visibility was the most appreciated feature.
- Mouse pen control is awkward in flat-screen mode (expected for desktop/testing use).
- HSV color picker intuitive and easy to use.

EVALUATION

Evaluation & Results

Formal Evaluation: SUS Usability & SUS Presence



SUS Usability

Participant	Mean score
P1	3.50
P2	3.17
P3	4.23
P4	3.17
Mean	**3.42**

SUS Presence

CONCLUSIONS

Conclusions & Future Work

Key Results

- Main goal achieved: presenter handoffs eliminated.
- Built a collaborative, social, multi-user architecture on Unity 6 + Ubiq with open-source tools.
- Up to 4 simultaneous users: positive formal tests, immersive experience.
- Deliberate simplification (whiteboard vs. desktop stream), maintainable for future versions.

Future Work

- Proper disconnect handling: UUID ⟶ slot mapping
- More formal study with Meta Quest 3 hardware
- Support more than 4 users with a dynamic workstation layout
- Replace whiteboard with a real desktop video stream (Unity screen capture)